Card of Stone

คุณได้รับสำรับไพ่จำนวน N ใบ โดยไพ่แต่ละใบจะถูกกำหนดด้วยแต้ม (rank) และดอก (suit) ดังต่อไปนี้:

- **แต้ม (Rank)** เรียงลำดับจากน้อยไปมาก: `ace < two < three < four < five < six < seven < eight < nine < ten < jack < queen < king`
- **ดอก (Suit)** เรียงลำดับจากน้อยไปมาก: `clubs < diamonds < hearts < spades`

ไพ่ A จะถือว่ามีค่ามากกว่าไพ่ B ก็ต่อเมื่อ:

- แต้มของ A มากกว่าแต้มของ B หรือ
- แต้มของ A เท่ากับแต้มของ B แต่ดอกของ A สูงกว่าดอกของ B

ไพ่แต่ละใบสามารถชี้ไปยังไพ่ใบถัดไปได้**สูงสุด 1 ใบ**ผ่านการเรียกเมธอด `card::add(card &next_card)`

รายละเอียดการเขียนโปรแกรม

คุณต้องเขียนเมธอด (Method) และฟังก์ชัน (Function) ทั้งหมดลงในไฟล์ `card.cpp` ภายใต้เนมสเปซ `namespace stone` ซึ่งประกอบด้วยโครงสร้างข้อมูลและส่วนประกอบดังต่อไปนี้:

1. ชนิดข้อมูลแจงนับ (Enumerations)

```
enum class rank {
    ace,  two, three, four,
    five, six, seven, eight,
    nine, ten, jack,  queen, king
};
enum class suit { clubs, diamonds, hearts, spades };
```

- `enum class rank`: กำหนดแต้มของไพ่ทั้งหมด 13 ระดับ เรียงตามค่าลำดับจากน้อยไปมาก ได้แก่:
- `enum class suit`: กำหนดดอกของไพ่ทั้งหมด 4 ดอก เรียงตามค่าลำดับจากน้อยไปมาก

2. ชนิดข้อมูลคลาส (Class)

```
class card {
public:
    suit m_suit;
    rank m_rank;
    card *m_next = nullptr;
```

```
    card();
    card(rank new_rank, suit new_suit);
    ~card();

    void add(card &next_card);
};
```

- `m_rank`: ชนิดข้อมูล rank ใช้เก็บค่าแต้มของไพ่ใบนี้
- `m_suit`: ชนิดข้อมูล suit ใช้เก็บค่าดอกของไพ่ใบนี้
- `m_next`: พอยน์เตอร์ชนิด `card *` ชี้ไปยังไพ่ใบถัดไป (ค่าเริ่มต้นคือ `nullptr`)
- `card()`: คอนสตรัคเตอร์ปริยาย
- `card(rank new_rank, suit new_suit)`: กำหนดค่าแต้ม `new_rank` และดอก `new_suit` ให้แก่ไพ่
- `~card()`: ดีสตรัคเตอร์สำหรับคืนทรัพยากรเมื่อสิ้นสุดการใช้งาน
- `void add(card &next_card)`: กำหนดให้พอยน์เตอร์ `m_next` ของไพ่ใบปัจจุบันชี้ไปยัง `next_card` (สร้างเส้นทางเชื่อมโยงทิศทางเดียวจากไพ่ปัจจุบันไปยังไพ่เป้าหมาย) หากมีการเรียกเมธอด `add` ซ้ำมากกว่าหนึ่งครั้งบนไพ่ใบเดิม ค่าของ `m_next` **จะถูกเขียนทับด้วยการเชื่อมโยงล่าสุด**

3. ฟังก์ชัน (Function)

```
int answer(std::vector<card> &cards);
```

- `cards`: เวกเตอร์ที่เก็บอ็อบเจกต์ `card` ทั้งหมดจำนวน N ใบในชุดคำถามนั้น ซึ่งเกรดเดอร์ได้กำหนดแต้มดอก และเรียก `add()` เพื่อเชื่อมโยงความสัมพันธ์เรียบร้อยแล้ว
- เงื่อนไขการทำงาน: สามารถเริ่มต้นการเดินทางที่ไพ่ใบใดก็ได้ในเวกเตอร์ สามารถเดินจากไพ่ปัจจุบัน `u` ไปยังไพ่ใบถัดไป `v` ผ่าน pointer `u.m_next` ได้ก็ต่อเมื่อ `u.m_next == nullptr` และไพ่ `v` มีค่ามากกว่า ไพ่ `u` **อย่างเคร่งครัด (ตามลำดับแต้มและดอกที่กำหนด)** หากไพ่ถัดไปมีค่าน้อยกว่าหรือเท่าเดิม จะไม่สามารถเดินต่อไปได้ (สิ้นสุดเส้นทางที่ไพ่ `u`)
- ส่งคืนค่าจำนวนเต็ม **แสดงถึงจำนวนไพ่ที่มากที่สุดบนเส้นทางการเดินที่ถูกต้องตามเงื่อนไข** (ความยาวเส้นทางนับตามจำนวนโหนดไพ่ โดยเส้นทางที่มีไพ่ใบเดียวจะมีความยาวเท่ากับ 1)

ขอบเขต

- $1 \le Q \le 100{,}000$
- $1 \le N \le 100{,}000$
- $1 \le N\,Q \le 1{,}000{,}000$

เกรดเดอร์ตัวอย่าง

เกรดเดอร์ตัวอย่างจะอ่านข้อมูลดังต่อไปนี้:

```
Q
N[0]
rank[0][0] suit[0][0]
...
rank[0][N[0]-1] suit[0][N[0]-1]
K[0]
curr[0][0] next[0][0]
...
curr[0][K-1] next[0][K-1]
...
curr[0][0] next[0][0]
...
curr[0][K-1] next[0][K-1]
...
N[1]
...
(ทำซ้ำจนครบ Q ชุดคำถาม)
```

บรรทัดแรก: จำนวนเต็ม Q แสดงถึงจำนวนชุดคำถามทั้งหมด

- ในแต่ละชุดคำถามที่ $0 \le i < Q$
- บรรทัดแรกของชุดคำถาม ระบุจำนวนเต็ม $N[i]$ แทนจำนวนไพ่ทั้งหมด
- อีก $N[i]$ บรรทัดถัดมา แต่ละบรรทัดประกอบด้วยข้อความ 2 ค่าคั่นด้วยเว้นวรรค ได้แก่ $rank[i][j]$ และ $suit[i][j]$ แสดงแต้มและดอกของไพ่ดัชนีที่ j $(0 \le j < N[i])$
- บรรทัดถัดมา ระบุจำนวนเต็ม $K[i]$แทนจำนวนการเชื่อมโยงของไพ่
- อีก $K[i]$ บรรทัดถัดมา แต่ละบรรทัดประกอบด้วยจำนวนเต็ม 2 จำนวน ได้แก่ $curr[i][j]$ และ $next[i][j]$ $(0 \le curr[i][j], next[i][j] < N)$ สำหรับทุก $(0 \le j < K[i])$ แสดงคำสั่งการเรียกฟังก์ชัน `cards[curr].add(cards[next])` ตามลำดับ เพื่อให้ไพ่ดัชนีที่ `curr` ชี้ไปยังไพ่ดัชนีที่ `next`

เกรดเดอร์ตัวอย่างจะแสดงผลดังต่อไปนี้:

```
S[0]
S[1]
...
S[Q-1]
```

$S[k]$ $(0 \leq k < Q)$ แสดงถึงความยาวของเส้นทางไพ่ที่ยาวที่สุดของชุดคำถามที่ k

ตัวอย่างข้อมูลนำเข้าและข้อมูลส่งออก

ตัวอย่างข้อมูลนำเข้า

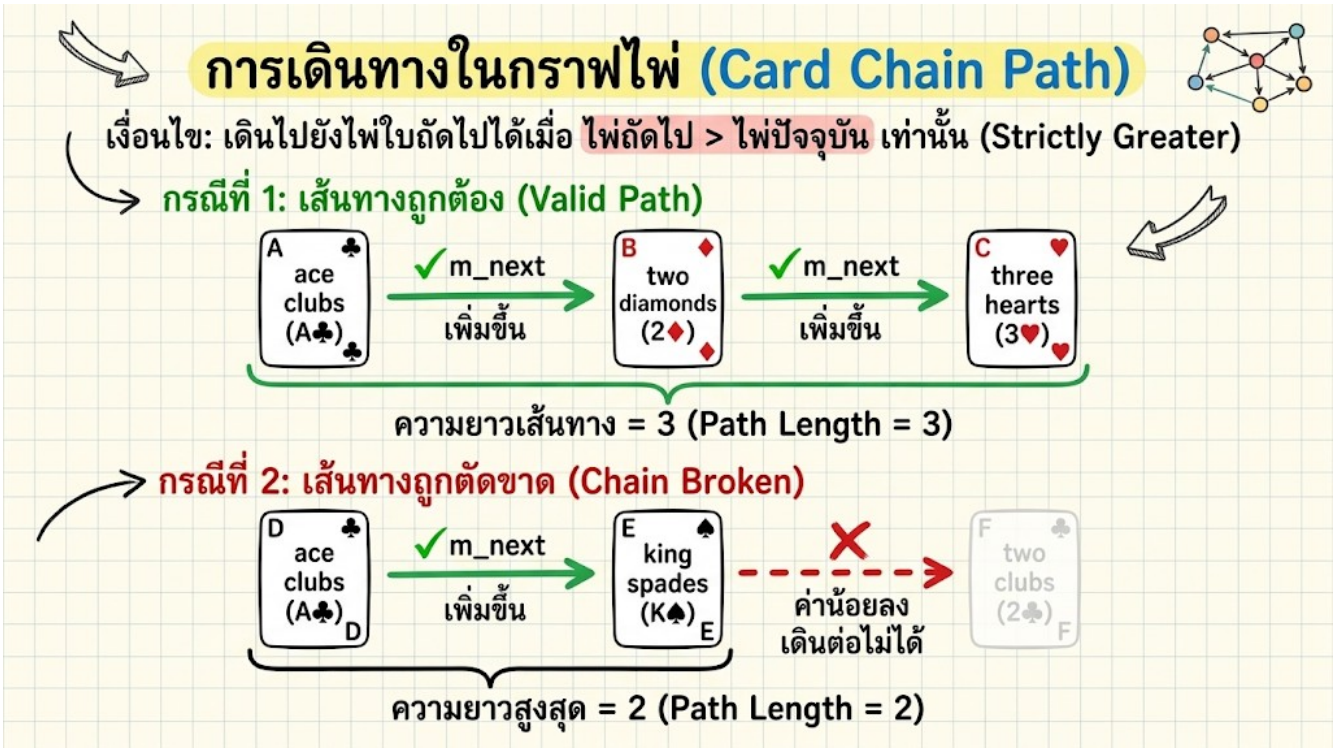
```
2
3
ace clubs
two diamonds
three hearts
2
0 1
1 2
3
ace clubs
king spades
two clubs
2
0 1
1 2
```

ตัวอย่างข้อมูลส่งออก

```
3
2
```

คำอธิบายตัวอย่าง:

- **ชุดคำถามที่ 1:** ไพ่มีลำดับ `ace clubs → two diamonds → three hearts` ซึ่งมีค่าเพิ่มขึ้นอย่างเคร่งครัดตลอดเส้นทาง จึงเดินทางได้ครบทั้ง 3 ใบ ตอบ 3
- **ชุดคำถามที่ 2:** การเดินจาก `ace clubs → king spades` ถูกต้องแต่จาก `king spades → two clubs` มีค่าน้อยลง จึงไม่สามารถเดินต่อได้ เส้นทางที่ยาวที่สุดได้ความยาว 2



ข้อจำกัด

- Time limit: 1.0 seconds
- Memory limit: 256 MB